École Marocaine des Sciences de l'Ingénieur
EMSI – Rabat

Filière : Génie Informatique
Classe : 4IIR13

Rapport de Projet

Mise en place d'un accès sudo sécurisé sous Linux

Réalisé par :

Amine El Ouali

Med Amine Halhoul

Ayman Dahraoui

Encadré par :

Pr.Mohammed Roudani

Année universitaire : 2025 – 2026

Résumé

Ce projet porte sur la mise en place d'un mécanisme de contrôle d'accès basé sur les rôles (*Role-Based Access Control – RBAC*) dans un environnement Linux, en s'appuyant sur l'outil `sudo`. L'objectif principal est de renforcer la sécurité du système en appliquant le principe du moindre privilège et en limitant les risques liés à l'escalade de privilèges.

La solution proposée repose sur la définition de rôles distincts, à savoir *Technicien support*, *Administrateur applicatif* et *NOC (Centre d'Opérations Réseau)*, chacun disposant d'autorisations spécifiques et strictement contrôlées. Les privilèges sont attribués au niveau des groupes afin de simplifier l'administration et de garantir une meilleure cohérence avec le modèle RBAC.

L'implémentation a été réalisée sur une plateforme Red Hat Enterprise Linux, en utilisant les fichiers `/etc/sudoers.d` et les alias `Cmnd_Alias` pour structurer les règles d'accès. Des mécanismes de journalisation et d'audit ont été intégrés afin d'assurer la traçabilité des actions exécutées avec des privilèges élevés.

Les tests effectués démontrent que les commandes autorisées pour chaque rôle sont correctement exécutées, tandis que les commandes non autorisées sont systématiquement refusées. Cette approche permet ainsi d'améliorer significativement la sécurité, la traçabilité et la gestion des accès privilégiés dans un environnement Linux.

Table des figures

4.1 Ajout d'utilisateurs aux groupes `techsupport` et `adminapp` 13
4.2 Ajout d'utilisateurs au groupe `NOC` 14
4.3 Règles pour le Technicien support 15
4.4 Règles pour l'Administrateur applicatif 16
4.5 Règles pour le NOC 16
4.6 Afficher les commandes sudo d'utilisateur Elouali du groupe adminapp 17
4.7 Afficher les commandes sudo d'utilisateur Mehdi du groupe noc 17
4.8 Afficher les commandes sudo pour la session courante 17

5.1 Tests du Technicien support 18
5.2 Tests de l'Administrateur applicatif 19
5.3 Tests du rôle NOC 19

Table des matières

1 Introduction générale **5**
1.1 Contexte du projet 5
1.2 Objectifs pédagogiques et techniques 5
1.3 Méthodologie de travail 5

2 Contexte sécurité et problématique **7**
2.1 Problèmes d'escalade de privilèges 7
2.2 Principe du moindre privilège 7
2.2.1 Définition et enjeux 7
2.2.2 Application dans les systèmes Linux 8
2.3 Approche basée sur les rôles (RBAC) 8
2.3.1 Avantages de la séparation des rôles 8
2.3.2 Objectifs de la solution proposée 9

3 Définition des rôles et besoins d'accès **10**
3.1 Présentation des rôles 10
3.1.1 Objectifs de la séparation des rôles 10
3.2 Rôle : Technicien support 10
3.2.1 Objectifs 10
3.2.2 Commandes autorisées 10
3.3 Rôle : Administrateur applicatif 11
3.3.1 Objectifs 11
3.3.2 Services concernés 11
3.3.3 Limites d'accès 11
3.4 Rôle : NOC (Centre d'Opérations Réseau) 11
3.4.1 Objectifs 11
3.4.2 Commandes de surveillance 12
3.4.3 Restrictions appliquées 12

4 Implémentation de la délégation sudo **13**
4.1 Création des groupes et des utilisateurs 13
4.2 Organisation des fichiers sudoers 14
4.2.1 Présentation du fichier sudoers 14
4.2.2 Qu'est-ce que visudo ? 14
4.2.3 Utilisation du répertoire `/etc/sudoers.d` 14
4.2.4 Modification des règles sudo 15
4.3 Définition des règles sudo 15
4.3.1 Règles pour le Technicien support 15
4.3.2 Règles pour l'Administrateur applicatif 16

4.3.3 Règles pour le NOC 16
4.4 Journalisation et audit 16

5 Tests et validation **18**
5.1 Tests par rôle 18
5.1.1 Tests du Technicien support 18
5.1.2 Tests de l'Administrateur applicatif 19
5.1.3 Tests du rôle NOC 19
5.2 Analyse des résultats 19

6 Conclusion et perspectives **20**
6.1 Limites de la solution 20
6.2 Améliorations possibles 20

Chapitre 1

Introduction générale

1.1 Contexte du projet

Dans un contexte où les systèmes Linux sont largement utilisés au sein des infrastructures informatiques des entreprises, la gestion sécurisée des privilèges constitue un enjeu majeur de cybersécurité. L'utilisation de mécanismes tels que `sudo` permet à des utilisateurs non privilégiés d'exécuter certaines commandes avec des droits élevés. Toutefois, une mauvaise configuration de ces accès peut entraîner des risques importants, notamment des escalades de privilèges, des erreurs humaines ou des compromissions du système.

Ce projet s'inscrit dans le cadre de l'administration sécurisée des systèmes Red Hat Enterprise Linux (RHEL) et vise à mettre en œuvre une délégation des privilèges basée sur les rôles, conforme aux bonnes pratiques de sécurité et au principe du moindre privilège.

1.2 Objectifs pédagogiques et techniques

L'objectif principal de ce projet est de renforcer la compréhension et la maîtrise des mécanismes de gestion des privilèges sous Linux à travers une approche pratique et orientée cybersécurité.

Sur le plan pédagogique, ce travail vise à :

— Comprendre les risques liés à une utilisation non maîtrisée de `sudo`
— Appliquer le principe du moindre privilège dans un environnement Linux
— Mettre en œuvre un contrôle d'accès basé sur les rôles (RBAC)

Sur le plan technique, les objectifs sont :

— Créer des groupes représentant des rôles métier
— Définir des règles `sudo` précises et sécurisées
— Mettre en place la journalisation et l'audit des actions privilégiées
— Tester et valider les accès attribués à chaque rôle

1.3 Méthodologie de travail

La méthodologie adoptée repose sur une approche progressive et structurée. Dans un premier temps, une analyse des problématiques de sécurité liées aux privilèges administratifs est réalisée. Ensuite, les rôles métier sont définis en fonction des besoins réels d'accès, puis traduits en règles `sudo` précises.

L'implémentation est effectuée dans un environnement Red Hat Enterprise Linux de test, permettant de valider les configurations sans impacter un système de production. Enfin, des tests fonctionnels et de sécurité sont réalisés afin de vérifier le respect des restrictions définies et l'efficacité des mécanismes de journalisation.

Chapitre 2

Contexte sécurité et problématique

2.1 Problèmes d'escalade de privilèges

L'élévation des privilèges est le processus par lequel un acteur malveillant obtient un accès élevé et des droits administratifs à un système en exploitant des vulnérabilités de sécurité. En modifiant les autorisations d'identité pour leur accorder des droits accrus et des capacités administratives, l'élévation des privilèges permet aux attaquants d'effectuer des activités malveillantes avec un niveau de privilège plus élevé, ce qui peut potentiellement entraîner des dommages importants. Les systèmes informatiques disposent de différents niveaux de privilèges, allant des utilisateurs standard avec des autorisations limitées aux administrateurs ayant un contrôle complet sur le système. En cas de succès d'un incident d'élévation des privilèges, cela signifie qu'un attaquant a réussi à augmenter son propre niveau de privilège, obtenant ainsi un contrôle accru.

En retour, les cyberattaquants peuvent utiliser l'élévation des privilèges pour ouvrir de nouveaux vecteurs d'attaque sur un système cible, faisant évoluer le niveau de menace des simples infections par des logiciels malveillants à des violations de données catastrophiques et à des intrusions dans le réseau.

2.2 Principe du moindre privilège

2.2.1 Définition et enjeux

Le principe du moindre privilège fait référence à un concept de sécurité dans lequel on accorde à un utilisateur le niveau d'accès (ou les permissions) minimum requis pour accomplir son travail. Le principe du moindre privilège est largement considéré comme une bonne pratique de cybersécurité et constitue une étape fondamentale dans la protection des accès à privilèges aux données et actifs de forte valeur. Le principe du moindre privilège s'étend au-delà de l'accès des personnes.

Ce modèle peut s'appliquer aux applications, aux systèmes ou aux systèmes connectés qui requièrent des privilèges ou des permissions afin d'effectuer une tâche requise. L'application du moindre privilège garantit que l'outil non humain dispose uniquement de l'accès strictement nécessaire, et pas davantage.[2]

2.2.2 Application dans les systèmes Linux

Dans les systèmes Linux, le principe du moindre privilège est mis en œuvre à travers plusieurs mécanismes de sécurité intégrés au système d'exploitation. Le modèle de permissions basé sur les utilisateurs, les groupes et les droits d'accès (lecture, écriture et exécution) constitue la première ligne de défense.

L'utilisation de groupes permet de gérer les permissions de manière collective, en attribuant des droits à des ensembles d'utilisateurs partageant les mêmes fonctions. Les mécanismes avancés tels que les ACL (Access Control Lists) offrent un niveau de détail supplémentaire lorsque les permissions standards ne sont pas suffisantes. Par ailleurs, l'outil `sudo` joue un rôle central dans l'application du moindre privilège. Il permet de déléguer de manière contrôlée l'exécution de commandes spécifiques avec des privilèges élevés, sans accorder un accès administrateur complet. Cette délégation peut être strictement limitée par des règles précises, des groupes et des alias, renforçant ainsi la sécurité globale du système. Enfin, l'audit et la journalisation des actions privilégiées, notamment via `journalctl` et les logs `sudo`, contribuent à la traçabilité et à la détection d'activités suspectes, complétant ainsi l'application effective du principe du moindre privilège dans les environnements Linux.

2.3 Approche basée sur les rôles (RBAC)

Le contrôle d'accès basé sur les rôles ou RBAC (Role Based Access Control) désigne une méthode de gestion des accès des utilisateurs aux systèmes, réseaux ou ressources en fonction de leur rôle au sein d'une équipe ou d'une entreprise. Au lieu de configurer l'accès aux systèmes ou aux réseaux par utilisateur, le RBAC permet aux administrateurs informatiques de configurer un ensemble d'autorisations pour différents rôles, puis d'attribuer ces rôles aux utilisateurs en fonction de leur poste et du niveau d'accès dont ils ont besoin. Les équipes peuvent ainsi ajouter, modifier et supprimer facilement des autorisations pour l'ensemble des utilisateurs d'un groupe ayant le même rôle, ou modifier rapidement le niveau d'accès d'un seul utilisateur en changeant ou en supprimant son rôle. Tous les systèmes de contrôle d'accès basé sur les rôles suivent les mêmes principes de base :

— Chaque utilisateur reçoit un ou plusieurs rôles.
— Des autorisations sont définies par rôle utilisateur.
— Les utilisateurs obtiennent les autorisations lorsqu'ils sont actifs dans un rôle.[1]

2.3.1 Avantages de la séparation des rôles

La séparation des rôles constitue un principe fondamental en cybersécurité et en administration des systèmes. Elle permet de réduire significativement les risques liés à une mauvaise utilisation des privilèges et d'améliorer la gouvernance des accès.

Tout d'abord, cette approche permet de limiter l'exposition du système aux attaques internes ou aux erreurs humaines. En attribuant uniquement les permissions nécessaires à chaque fonction métier, l'impact potentiel d'un compte compromis est fortement réduit. Ensuite, la séparation des rôles facilite l'application du principe du moindre privilège. Chaque utilisateur dispose exclusivement des droits requis pour accomplir ses missions, sans accès inutile aux fonctions sensibles du système, telles que l'administration globale ou la modification de la configuration critique. Par ailleurs, cette organisation améliore la traçabilité et l'audit des actions administratives. Les opérations effectuées via `sudo` peuvent être associées à un rôle précis, ce qui simplifie l'analyse des journaux et la détection d'activités anormales ou non autorisées.

Enfin, la séparation des rôles renforce la maintenabilité et l'évolutivité du système. Les changements d'organisation, tels que l'arrivée ou le départ d'un collaborateur, peuvent être gérés simplement en modifiant son appartenance à un groupe, sans devoir reconfigurer individuellement les permissions.

2.3.2 Objectifs de la solution proposée

Pour répondre à ces problématiques, une approche basée sur les rôles a été adoptée dans ce projet. Chaque rôle correspond à un groupe d'utilisateurs disposant de permissions spécifiques et clairement définies. Plutôt que d'attribuer des droits individuellement à chaque utilisateur, les autorisations `sudo` sont associées à des groupes représentant des fonctions métier, telles que *Technicien support*, *Administrateur applicatif* et *NOC (Centre d'Opérations Réseau)* .

Chapitre 3

Définition des rôles et besoins d'accès

3.1 Présentation des rôles

Dans une démarche de sécurisation des accès administratifs sous Linux, la définition claire des rôles constitue une étape essentielle. Chaque rôle correspond à une fonction métier précise et dispose uniquement des privilèges nécessaires à l'exécution de ses missions, conformément au principe du moindre privilège.

Dans ce projet, la gestion des accès repose sur une approche RBAC (Role Based Access Control), où les permissions `sudo` sont attribuées à des groupes représentant des rôles métiers.

3.1.1 Objectifs de la séparation des rôles

La séparation des rôles vise à :

— Limiter les risques d'escalade de privilèges
— Réduire l'impact d'une compromission de compte
— Faciliter l'administration et l'audit des accès
— Appliquer strictement le principe du moindre privilège

Trois rôles principaux ont été définis : *Technicien support*, *Administrateur applicatif* et *NOC (Centre d'Opérations Réseau)*.

3.2 Rôle : Technicien support

3.2.1 Objectifs

Le rôle de *Technicien support* assure le support technique de premier niveau. Il est chargé de diagnostiquer les incidents, de vérifier l'état des services et de consulter les journaux système afin d'identifier les causes des dysfonctionnements. Aucune action de modification ou d'administration n'est autorisée pour ce rôle.

3.2.2 Commandes autorisées

Les commandes autorisées pour ce rôle sont strictement limitées à la consultation.

— `systemctl status <service>` Permet de consulter l'état détaillé d'un service (actif, inactif, échec) ainsi que les derniers messages associés.
— `systemctl is-active <service>` Indique si un service est actuellement en cours d'exécution. Cette commande est utilisée pour un diagnostic rapide.

— `systemctl is-enabled <service>` Permet de vérifier si un service est configuré pour démarrer automatiquement au démarrage du système.
— `journalctl` Affiche les journaux système afin d'analyser les événements récents.
— `journalctl -u <service>` Permet de consulter les journaux associés à un service spécifique.
— `journalctl -p err` Filtre les journaux pour afficher uniquement les messages de niveau erreur, facilitant l'identification des incidents critiques.

Ces commandes offrent une capacité de diagnostic complète sans permettre de modifier le comportement du système.

3.3 Rôle : Administrateur applicatif

3.3.1 Objectifs

Le rôle d'*Administrateur applicatif* est responsable de la gestion opérationnelle des applications métier. Il intervient lors des opérations de maintenance, de mise à jour ou de rechargement de configuration applicative.

3.3.2 Services concernés

Ce rôle est limité aux services applicatifs spécifiques, notamment :
— `custom-service.service`

Aucun accès n'est accordé aux services critiques du système.

3.3.3 Limites d'accès

Les commandes autorisées sont les suivantes :
— `systemctl status custom-service.service` Permet de vérifier l'état du service applicatif.
— `systemctl restart custom-service.service` Permet de redémarrer le service après une mise à jour ou un incident.
— `systemctl reload custom-service.service` Permet de recharger la configuration du service sans interruption complète.
— `journalctl -u custom-service.service` Permet d'analyser les journaux liés exclusivement au service applicatif.

L'utilisation de l'option `NOPASSWD` est justifiée par le caractère opérationnel des interventions, tout en restant limitée à un périmètre strictement défini.

3.4 Rôle : NOC (Centre d'Opérations Réseau)

3.4.1 Objectifs

Le rôle *NOC* est chargé de la surveillance continue de l'infrastructure système et réseau. Il assure une visibilité globale sur l'état des ressources afin de détecter rapidement les anomalies.

3.4.2 Commandes de surveillance

Les commandes autorisées sont exclusivement des commandes de monitoring :

— `df` Permet de vérifier l'utilisation de l'espace disque et d'anticiper les risques de saturation.
— `free` Affiche l'état de la mémoire physique et de la mémoire swap.
— `uptime` Indique la durée de fonctionnement du système et la charge moyenne.
— `ss` Fournit des informations sur les connexions réseau actives et les ports en écoute.

3.4.3 Restrictions appliquées

Le rôle NOC ne dispose d'aucune commande permettant de modifier la configuration, d'arrêter des services ou d'exécuter des actions administratives. Les accès sont strictement limités à la consultation, garantissant ainsi une surveillance efficace sans risque d'altération du système.

Chapitre 4

Implémentation de la délégation sudo

4.1 Création des groupes et des utilisateurs

Un utilisateur représente une identité unique sur le système. Il peut être un humain, un script ou un service. Chaque utilisateur a son propre espace de travail (répertoire personnel), ses fichiers et ses droits.

Un groupe, lui, rassemble plusieurs utilisateurs qui partagent les mêmes permissions sur des fichiers ou des ressources. Plutôt que de gérer les droits individuellement, on les attribue à un groupe : simple, propre et scalable.

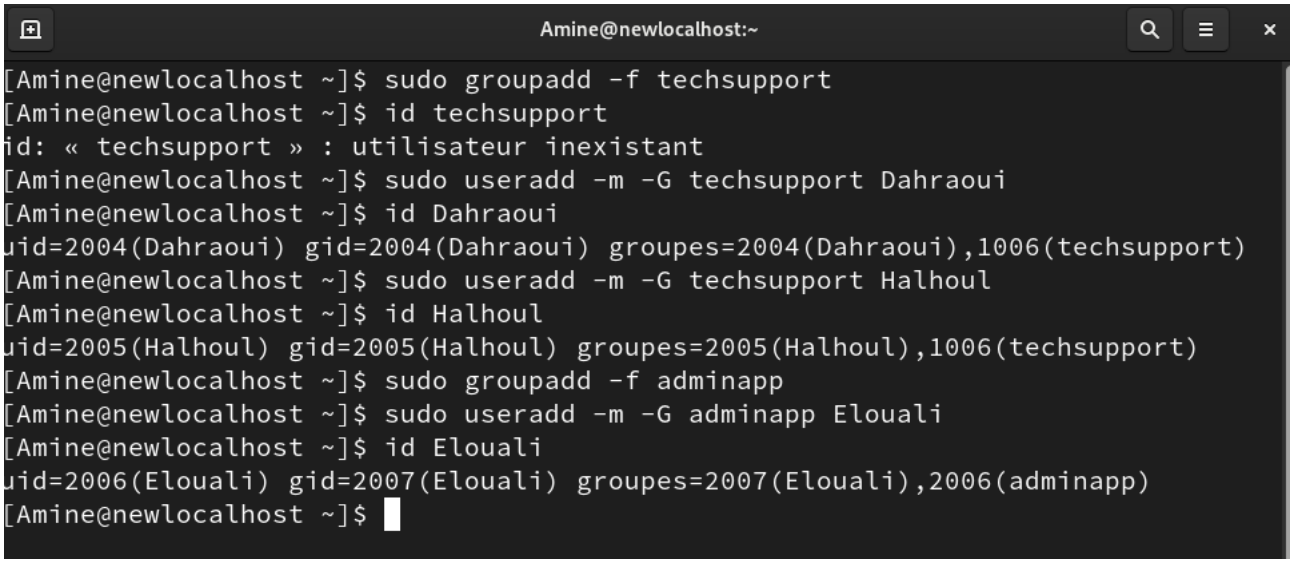

FIGURE 4.1 – Ajout d'utilisateurs aux groupes `techsupport` et `adminapp`

```
[Amine@newlocalhost ~]$ sudo groupadd -f noc
[sudo] Mot de passe de Amine :
[Amine@newlocalhost ~]$ sudo useradd -m -G Mehdi
useradd: group 'Mehdi' does not exist
[Amine@newlocalhost ~]$ sudo useradd -m -G noc Mehdi
[Amine@newlocalhost ~]$ sudo useradd -m -G noc Anas
[Amine@newlocalhost ~]$ id Anas
uid=2008(Anas) gid=2010(Anas) groupes=2010(Anas),2008(noc)
[Amine@newlocalhost ~]$ id Mehdi
uid=2007(Mehdi) gid=2009(Mehdi) groupes=2009(Mehdi),2008(noc)
```

FIGURE 4.2 – Ajout d'utilisateurs au groupe `NOC`

4.2 Organisation des fichiers sudoers

4.2.1 Présentation du fichier sudoers

Le fichier /etc/sudoers est le point de contrôle principal de la gestion des privilèges sudo sur un système Linux . Il définit quels utilisateurs peuvent exécuter quelles commandes en tant que superutilisateur (root) ou d'autres comptes. Comme le souligne DigitalOcean, modifier ce fichier est une opération critique : une erreur de syntaxe peut vous verrouiller hors du système ou créer des failles de sécurité . La seule manière sûre recommandée d'y apporter des changements est d'utiliser la commande visudo , qui vérifie automatiquement la syntaxe avant de valider les modifications . [3]

4.2.2 Qu'est-ce que visudo ?

La configuration de la commande sudo se fait par le biais d'un fichier situé dans /etc/sudoers. Étant donné que l'utilisation d'une mauvaise syntaxe dans le fichier /etc/sudoers vous laissera créer un système défaillant dans lequel il sera impossible d'obtenir des privilèges élevés, il est important d'utiliser la commande `visudo` pour modifier le fichier. La commande `visudo` ouvre un éditeur de texte normalement, mais valide la syntaxe du fichier au moment de la sauvegarde. Cela empêche les erreurs de configuration de venir bloquer les opérations sudo, qui est votre seule façon d'obtenir des privilèges root. Traditionnellement, visudo ouvre le fichier /etc/sudoers avec l'éditeur de texte vi. Cependant, en Ubuntu, visudo est configuré pour utiliser l'éditeur de texte nano à la place.[3]

4.2.3 Utilisation du répertoire `/etc/sudoers.d`

Le répertoire /etc/sudoers.d/ permet de stocker des fichiers de configuration sudo personnalisés, offrant une manière plus modulaire et maintenable d'ajouter des règles sans modifier directement le fichier /etc/sudoers principal. Les fichiers dans ce répertoire sont traités par sudo par ordre alphabétique et numérique, et doivent être édités avec sudo visudo -f /chemin/vers/fichier pour garantir une vérification de la syntaxe et éviter le blocage du système. Avantages de /etc/sudoers.d/ :

Modularité : Sépare les configurations personnalisées de la configuration par défaut.

Maintien : Facilite la gestion des permissions, notamment lors des mises à jour système (évite les conflits).

Réversibilité : En cas de problème, il suffit de supprimer le fichier fautif pour revenir à la configuration précédente. [3]

4.2.4 Modification des règles sudo

La modification des règles `sudo` doit être réalisée avec précaution, car une erreur de syntaxe dans le fichier `/etc/sudoers` peut entraîner une perte totale de l'accès administrateur au système. Pour éviter ce type de problème, l'outil `visudo` est utilisé. Celui-ci vérifie automatiquement la syntaxe avant d'enregistrer les modifications et empêche l'enregistrement d'un fichier invalide.

Dans ce projet, les règles `sudo` ne sont pas définies directement dans le fichier principal `/etc/sudoers`, mais dans des fichiers dédiés situés dans le répertoire `/etc/sudoers.d`. Cette approche améliore la lisibilité, facilite la maintenance et respecte les bonnes pratiques recommandées par Red Hat. Les privilèges `sudo` sont attribués aux **groupes** plutôt qu'aux utilisateurs individuels afin de respecter le modèle de contrôle d'accès basé sur les rôles (RBAC).

4.3 Définition des règles sudo

4.3.1 Règles pour le Technicien support

```
[Amine@newlocalhost ~]$ sudo visudo /etc/sudoers.d/techsupport
[Amine@newlocalhost ~]$ sudo cat /etc/sudoers.d/techsupport
Cmnd_Alias TS_SYSTEMCTL = \
    /usr/bin/systemctl status *, \
    /usr/bin/systemctl is-active *, \
    /usr/bin/systemctl is-enabled *

Cmnd_Alias TS_LOGS = \
    /usr/bin/journalctl, \
    /usr/bin/journalctl -u *, \
    /usr/bin/journalctl -p err

%techsupport ALL=(root) TS_SYSTEMCTL, TS_LOGS
[Amine@newlocalhost ~]$
```

FIGURE 4.3 – Règles pour le Technicien support

4.3.2 Règles pour l'Administrateur applicatif

```
[Amine@newlocalhost ~]$ sudo visudo /etc/sudoers.d/adminapp
[Amine@newlocalhost ~]$ sudo cat /etc/sudoers.d/adminapp
[sudo] Mot de passe de Amine :
Cmnd_Alias APP_SERVICE_STATUS = \
    /usr/bin/systemctl status custom-service.service

Cmnd_Alias APP_SERVICE_CONTROL = \
    /usr/bin/systemctl restart custom-service.service, \
    /usr/bin/systemctl reload custom-service.service

Cmnd_Alias APP_LOGS = \
    /usr/bin/journalctl -u custom-service.service

%adminapp ALL=(root) NOPASSWD: APP_SERVICE_STATUS, APP_SERVICE_CONTROL, APP_LOGS
[Amine@newlocalhost ~]$
```

FIGURE 4.4 – Règles pour l'Administrateur applicatif

4.3.3 Règles pour le NOC

```
[Amine@newlocalhost ~]$ sudo visudo -f /etc/sudoers.d/noc
[Amine@newlocalhost ~]$ sudo cat /etc/sudoers.d/noc
Cmnd_Alias NOC_SYSTEM_INFO = \
    /usr/bin/df, \
    /usr/bin/free, \
    /usr/bin/uptime, \
    /usr/bin/ss
%noc ALL=(root) NOC_SYSTEM_INFO
[Amine@newlocalhost ~]$
```

FIGURE 4.5 – Règles pour le NOC

4.4 Journalisation et audit

Les outils `journalctl` et `auditd` sont utilisés pour analyser les actions à privilèges élevés et répondre aux exigences de sécurité et d'audit.

```
[Amine@newlocalhost ~]$ sudo cat /var/log/secure | grep Elouali
[sudo] Mot de passe de Amine :
Jan  5 20:01:38 newlocalhost unix_chkpwd[3124]: password check failed for user (Elouali)
Jan  5 20:01:38 newlocalhost su[3114]: pam_unix(su-l:auth): authentication failure; logname=Amine uid=1
001 euid=0 tty=/dev/pts/0 ruser=Amine rhost=  user=Elouali
Jan  5 20:01:50 newlocalhost su[3133]: pam_unix(su-l:session): session opened for user Elouali(uid=2006
) by Amine(uid=1001)
Jan  5 20:04:03 newlocalhost sudo[3235]: Elouali : command not allowed ; TTY=pts/0 ; PWD=/home/Elouali
; USER=root ; COMMAND=/bin/journalctl -xe
Jan  5 20:05:43 newlocalhost sudo[3272]: Elouali : command not allowed ; TTY=pts/0 ; PWD=/home/Elouali
; USER=root ; COMMAND=/bin/cat /var/log/secure
Jan  5 20:05:51 newlocalhost su[3133]: pam_unix(su-l:session): session closed for user Elouali
[Amine@newlocalhost ~]$
```

FIGURE 4.6 – Afficher les commandes sudo d'utilisateur Elouali du groupe adminapp

```
[Amine@newlocalhost ~]$ sudo journalctl _COMM=sudo | grep Mehdi
[sudo] Mot de passe de Amine :
janv. 05 20:17:00 newlocalhost sudo[3611]:    Mehdi : command not allowed ; TTY=pts/0 ; PWD=/home/Mehdi
 ; USER=root ; COMMAND=/bin/journalctl _COMM=sudo
[Amine@newlocalhost ~]$ cat /var/log/secure | grep Mehdi
cat: /var/log/secure: Permission non accordée
[Amine@newlocalhost ~]$ sudo cat /var/log/secure | grep Mehdi
Jan  5 20:15:00 newlocalhost su[3492]: pam_unix(su-l:session): session opened for user Mehdi(uid=2007)
by Amine(uid=1001)
Jan  5 20:15:15 newlocalhost su[3492]: pam_unix(su-l:session): session closed for user Mehdi
Jan  5 20:15:21 newlocalhost su[3546]: pam_unix(su-l:session): session opened for user Mehdi(uid=2007)
by Amine(uid=1001)
Jan  5 20:17:00 newlocalhost sudo[3611]:   Mehdi : command not allowed ; TTY=pts/0 ; PWD=/home/Mehdi ;
USER=root ; COMMAND=/bin/journalctl _COMM=sudo
Jan  5 20:17:23 newlocalhost su[3546]: pam_unix(su-l:session): session closed for user Mehdi
[Amine@newlocalhost ~]$
```

FIGURE 4.7 – Afficher les commandes sudo d'utilisateur Mehdi du groupe noc

```
[Amine@newlocalhost ~]$ sudo journalctl -xe | grep sudo
janv. 05 20:04:03 newlocalhost sudo[3235]:  Elouali : command not allowed ; TTY=pts/0 ; PWD=/home/Eloua
li ; USER=root ; COMMAND=/bin/journalctl -xe
janv. 05 20:05:43 newlocalhost sudo[3272]:  Elouali : command not allowed ; TTY=pts/0 ; PWD=/home/Eloua
li ; USER=root ; COMMAND=/bin/cat /var/log/secure
janv. 05 20:06:12 newlocalhost sudo[3295]:    Amine : TTY=pts/0 ; PWD=/home/Amine ; USER=root ; COMMAND
=/bin/cat /var/log/secure
janv. 05 20:06:12 newlocalhost sudo[3295]: pam_unix(sudo:session): session opened for user root(uid=0)
by Amine(uid=1001)
janv. 05 20:06:12 newlocalhost sudo[3295]: pam_unix(sudo:session): session closed for user root
janv. 05 20:17:00 newlocalhost sudo[3611]:    Mehdi : command not allowed ; TTY=pts/0 ; PWD=/home/Mehdi
 ; USER=root ; COMMAND=/bin/journalctl _COMM=sudo
janv. 05 20:17:41 newlocalhost sudo[3641]:    Amine : TTY=pts/0 ; PWD=/home/Amine ; USER=root ; COMMAND
=/bin/journalctl _COMM=sudo
janv. 05 20:17:41 newlocalhost sudo[3641]: pam_unix(sudo:session): session opened for user root(uid=0)
by Amine(uid=1001)
janv. 05 20:17:41 newlocalhost sudo[3641]: pam_unix(sudo:session): session closed for user root
janv. 05 20:18:48 newlocalhost sudo[3684]:    Amine : TTY=pts/0 ; PWD=/home/Amine ; USER=root ; COMMAND
=/bin/cat /var/log/secure
janv. 05 20:18:48 newlocalhost sudo[3684]: pam_unix(sudo:session): session opened for user root(uid=0)
by Amine(uid=1001)
janv. 05 20:18:48 newlocalhost sudo[3684]: pam_unix(sudo:session): session closed for user root
janv. 05 20:23:46 newlocalhost sudo[3773]:    Amine : TTY=pts/0 ; PWD=/home/Amine ; USER=root ; COMMAND
=/bin/journalctl -xe
janv. 05 20:23:46 newlocalhost sudo[3773]: pam_unix(sudo:session): session opened for user root(uid=0)
by Amine(uid=1001)
janv. 05 20:23:48 newlocalhost sudo[3773]: pam_unix(sudo:session): session closed for user root
janv. 05 20:26:26 newlocalhost sudo[3820]:    Amine : TTY=pts/0 ; PWD=/home/Amine ; USER=root ; COMMAND
```

FIGURE 4.8 – Afficher les commandes sudo pour la session courante

Chapitre 5

Tests et validation

5.1 Tests par rôle

5.1.1 Tests du Technicien support

```
[Amine@newlocalhost ~]$ su - Halhoul
Mot de passe :
[Halhoul@newlocalhost ~]$ sudo journalctl -p err

Nous espérons que vous avez reçu de votre administrateur système local
les consignes traditionnelles. Généralement, elles se concentrent sur ces trois éléments :

    #1) Respectez la vie privée des autres.
    #2) Réfléchissez avant d'utiliser le clavier.
    #3) De grands pouvoirs confèrent de grandes responsabilités.

[sudo] Mot de passe de Halhoul :
janv. 05 19:58:30 newlocalhost kernel: piix4_smbus 0000:00:07.3: SMBus base address uninitialized - up>
janv. 05 19:58:33 newlocalhost alsactl[1035]: alsa-lib main.c:1554:(snd_use_case_mgr_open) error: fail>
janv. 05 19:58:34 newlocalhost kernel: Warning: Unmaintained driver is detected: ip_set
janv. 05 19:58:35 newlocalhost systemd[1]: Failed to start Service personnalisé de test.
janv. 05 20:00:04 newlocalhost gdm-password][2259]: gkr-pam: unable to locate daemon control file
janv. 05 20:00:14 newlocalhost gdm-wayland-session[1474]: GLib: Source ID 2 was not found when attempt>
janv. 05 20:00:15 newlocalhost gdm-launch-environment][1290]: GLib-GObject: g_object_unref: assertion >
janv. 05 20:04:03 newlocalhost sudo[3235]:  Elouali : command not allowed ; TTY=pts/0 ; PWD=/home/Elou>
janv. 05 20:05:43 newlocalhost sudo[3272]:  Elouali : command not allowed ; TTY=pts/0 ; PWD=/home/Elou>
janv. 05 20:17:00 newlocalhost sudo[3611]:    Mehdi : command not allowed ; TTY=pts/0 ; PWD=/home/Mehd>
janv. 05 20:32:47 newlocalhost sudo[3968]: pam_unix(sudo:auth): conversation failed
janv. 05 20:32:47 newlocalhost sudo[3968]: pam_unix(sudo:auth): auth could not identify password for [>
```

FIGURE 5.1 – Tests du Technicien support

5.1.2 Tests de l'Administrateur applicatif

```
[Elouali@newlocalhost ~]$ systemctl status custom-service.service
● custom-service.service - Service personnalise
     Loaded: loaded (/etc/systemd/system/custom-service.service; enabled; prese>
     Active: active (running) since Mon 2026-01-05 19:58:34 +01; 4min 22s ago
   Main PID:      (bash)
      Tasks: 2 (limit: 22778)
     Memory: 640.0K
        CPU: 1.091s
     CGroup: [redacted]
```

FIGURE 5.2 – Tests de l'Administrateur applicatif

5.1.3 Tests du rôle NOC

```
[Amine@newlocalhost ~]$ su - Mehdi
Mot de passe :
[Mehdi@newlocalhost ~]$ free
               total        used        free      shared  buff/cache   available
Mem:         3709572     1495732     1730420       25912      751132     2213840
Swap:        4194300           0     4194300
[Mehdi@newlocalhost ~]$ uptime
 20:15:56 up 17 min,  2 users,  load average: 0,26, 0,15, 0,15
```

FIGURE 5.3 – Tests du rôle NOC

c

5.2 Analyse des résultats

Les résultats des tests confirment que la configuration de la délégation des privilèges via `sudo` est correctement appliquée pour l'ensemble des rôles définis.

Les commandes explicitement autorisées pour chaque rôle sont exécutées avec succès, sans erreurs ni élévation de privilèges non prévue. Chaque utilisateur peut ainsi accomplir les tâches correspondant à son rôle opérationnel.

À l'inverse, toute tentative d'exécution d'une commande non autorisée pour un rôle donné est systématiquement refusée par le mécanisme `sudo`. Ces tentatives sont bloquées avant exécution et enregistrées dans les journaux, garantissant la traçabilité et le contrôle des accès.

Cette validation démontre le respect du principe du moindre privilège ainsi que l'efficacité du modèle de contrôle d'accès basé sur les rôles (RBAC) mis en place.

Chapitre 6

Conclusion et perspectives

6.1 Limites de la solution

Bien que la solution de délégation des privilèges basée sur `sudo` et le modèle RBAC réponde efficacement aux objectifs de sécurité définis, certaines limites doivent être soulignées.

Tout d'abord, la gestion des règles `sudo` reste principalement manuelle. Dans un environnement de grande taille, comportant un nombre élevé d'utilisateurs et de rôles, cette approche peut devenir complexe à maintenir et sujette aux erreurs de configuration.

Ensuite, la solution repose sur des règles statiques. Les privilèges ne s'adaptent pas dynamiquement au contexte (heure, localisation, état du système), ce qui peut limiter le niveau de granularité du contrôle d'accès.

Enfin, l'audit est assuré principalement par les journaux `sudo` et `journalctl`. Bien que suffisants pour un environnement de test, ces mécanismes peuvent s'avérer insuffisants sans une centralisation ou une corrélation avancée des logs dans des environnements critiques.

6.2 Améliorations possibles

Plusieurs axes d'amélioration peuvent être envisagés afin de renforcer la sécurité et l'évolutivité de la solution. Une première amélioration consisterait à intégrer un système de gestion centralisée des identités et des accès, tel que LDAP ou FreeIPA, afin de simplifier l'administration des utilisateurs et des rôles à grande échelle.Il serait également pertinent d'enrichir la solution par l'utilisation de mécanismes d'audit avancés, comme `auditd` ou une plateforme SIEM, permettant une analyse centralisée et en temps réel des événements de sécurité.Par ailleurs, l'introduction de politiques de sécurité contextuelles, combinant `sudo` avec des outils comme SELinux, permettrait d'améliorer la protection contre les abus de privilèges et les attaques internes. Enfin, l'automatisation du déploiement et de la configuration (via Ansible ou scripts shell sécurisés) représenterait une évolution majeure pour garantir la cohérence et la reproductibilité de la configuration sur plusieurs systèmes.

Bibliographie

Bibliographie

[1] Comment modifier le fichier sudoers en toute sécurité. https://www.redhat.com/fr/topics/security/what-is-role-based-access-control.

[2] Qu'est-ce que le moindre privilège ? https://www.cyberark.com/fr/what-is/least-privilege/.

[3] Justin Ellingwood, Brian Boucheron et Vinayak Baranwal. Comment modifier le fichier sudoers en toute sécurité. https://www.digitalocean.com/community/tutorials/how-to-edit-the-sudoers-file-fr.